1. What is the primary 1-dimensional labeled array data structure in Pandas?

(A) DataFrame
(B) Series
(C) Panel
(D) Array1D

2. What is the primary 2-dimensional tabular labeled data structure in Pandas?

(A) Series
(B) DataFrame
(C) Matrix
(D) Tensor

3. Which function is used to read data from a Comma-Separated Values file into a DataFrame?

(A) pd.read_table()
(B) pd.read_csv()
(C) pd.open_csv()
(D) pd.load_csv()

4. Which method displays the first 5 rows of a DataFrame by default?

(A) df.top()
(B) df.head()
(C) df.first()
(D) df.preview()

5. Which method displays the last 5 rows of a DataFrame?

(A) df.bottom()
(B) df.tail()
(C) df.last()
(D) df.end()

6. Which method provides a concise summary of DataFrame info, memory usage, and column dtypes?

(A) df.summary()
(B) df.info()
(C) df.structure()
(D) df.glance()

7. Which method generates descriptive statistics including mean, std, min, and quartiles?

(A) df.stats()
(B) df.describe()
(C) df.metrics()
(D) df.values()

8. How do you select a single column named 'voltage' from a DataFrame `df`?

(A) `df['voltage']` or `df.voltage`
(B) `df.col('voltage')`
(C) `df.get_col('voltage')`
(D) `df[['voltage']]` only

9. What indexer is used for purely label-based selection in Pandas?

(A) .iloc
(B) .loc
(C) .ix
(D) .at_pos

10. What indexer is used for purely integer-position-based selection in Pandas?

(A) .loc
(B) .iloc
(C) .pos
(D) .index

11. What does `df.iloc[0]` return?

(A) The first column
(B) The first row as a Series
(C) The column headers
(D) The first element

12. What does `df.iloc[:, 0]` extract from a DataFrame?

(A) The first row
(B) The first column as a Series
(C) All rows and columns
(D) The index labels

13. Which function checks for missing (NaN or None) values across a DataFrame?

(A) df.is_empty()
(B) df.isna() or df.isnull()
(C) df.check_null()
(D) df.missing()

14. Which method drops rows containing missing values from a DataFrame?

(A) df.drop_missing()
(B) df.dropna()
(C) df.remove_na()
(D) df.purge_na()

15. Which method replaces missing NaN values with a specified value or strategy?

(A) df.replace_na()
(B) df.fillna()
(C) df.impute()
(D) df.set_default()

16. What attribute returns the dimensions of a DataFrame as a (rows, columns) tuple?

(A) df.size
(B) df.shape
(C) df.dim
(D) df.extent

17. What attribute returns the column names of a DataFrame?

(A) df.headers
(B) df.columns
(C) df.names
(D) df.keys_list

18. How do you export a DataFrame to a CSV file without saving the index column?

(A) df.to_csv('out.csv', index=False)
(B) df.save_csv('out.csv')
(C) df.write_csv('out.csv', no_index=True)
(D) df.export('out.csv')

19. Which method deletes a specific column named 'temp' along axis 1?

(A) df.remove('temp')
(B) df.drop('temp', axis=1)
(C) df.pop_col('temp')
(D) df.delete('temp')

20. What does the parameter `inplace=True` do in Pandas transformation methods?

(A) Creates a copy
(B) Modifies the DataFrame directly in-place and returns None
(C) Writes to disk
(D) Speeds up GPU

21. Which module from Matplotlib provides the standard state-machine plotting interface?

(A) matplotlib.plot
(B) matplotlib.pyplot
(C) matplotlib.pylab
(D) matplotlib.draw

22. Which command renders the generated plot figure onto the screen display?

(A) plt.render()
(B) plt.show()
(C) plt.display()
(D) plt.draw_now()

23. Which function sets the main title text of the current active plot?

(A) plt.heading()
(B) plt.title()
(C) plt.header()
(D) plt.main_text()

24. Which functions label the horizontal and vertical axes in Matplotlib?

(A) plt.x_axis() and plt.y_axis()
(B) plt.xlabel() and plt.ylabel()
(C) plt.set_x() and plt.set_y()
(D) plt.labels()

25. How do you enable gridlines on a plot?

(A) plt.enable_grid()
(B) plt.grid(True)
(C) plt.lines()
(D) plt.show_grid()

26. Which function adds a legend describing labeled plot curves?

(A) plt.keys()
(B) plt.legend()
(C) plt.description()
(D) plt.labels()

27. What shorthand color and linestyle code represents a red dashed line in `plt.plot()`?

(A) 'r--'
(B) 'red_dash'
(C) '--r'
(D) 'r-.'

28. Which function creates a scatter plot of individual coordinate markers?

(A) plt.points()
(B) plt.scatter()
(C) plt.dots()
(D) plt.markers()

29. Which function generates a vertical bar chart?

(A) plt.column()
(B) plt.bar()
(C) plt.histogram()
(D) plt.bars()

30. Which function generates a horizontal bar chart?

(A) plt.hbar()
(B) plt.barh()
(C) plt.bar_horizontal()
(D) plt.side_bar()

31. Which function plots a frequency distribution histogram of numerical data?

(A) plt.freq()
(B) plt.hist()
(C) plt.dist()
(D) plt.bins()

32. How do you specify the number of equal-width bins in a histogram?

(A) `plt.hist(data, bins=20)`
(B) `plt.hist(data, bars=20)`
(C) `plt.hist(data, slots=20)`
(D) `plt.hist(data, count=20)`

33. Which function saves the active figure to an image file (e.g. PNG, PDF, SVG)?

(A) plt.export()
(B) plt.savefig()
(C) plt.write_image()
(D) plt.dump()

34. What parameter in `plt.savefig()` controls the image resolution in dots per inch?

(A) resolution
(B) dpi
(C) scale
(D) quality

35. What parameter in `plt.savefig()` trims redundant exterior whitespace margins?

(A) `bbox_inches='tight'`
(B) `trim=True`
(C) `margin='none'`
(D) `crop=True`

36. Which function creates both a Figure and a grid of Subplot Axes in a single call?

(A) plt.create_grid()
(B) plt.subplots()
(C) plt.multi_axes()
(D) plt.layout()

37. What does `fig, (ax1, ax2) = plt.subplots(2, 1)` create?

(A) 2 side-by-side subplots
(B) 2 stacked subplots (2 rows, 1 column)
(C) A 2x2 grid
(D) Two separate figures

38. Which method on an Axes object sets the horizontal axis limits?

(A) ax.xlim()
(B) ax.set_xlim()
(C) ax.limit_x()
(D) ax.bounds_x()

39. Which method on an Axes object sets the vertical axis limits?

(A) ax.ylim()
(B) ax.set_ylim()
(C) ax.bounds_y()
(D) ax.range_y()

40. What does `ax.set_yscale('log')` do to a Matplotlib plot?

(A) Applies natural log to data values
(B) Changes the vertical axis scale to logarithmic (log10)
(C) Plots log error
(D) Inverts axis

41. What does `df.groupby('sensor_id')['reading'].mean()` compute?

(A) Mean of reading across whole dataframe
(B) Mean reading grouped separately for each unique sensor_id
(C) Sorts readings by sensor
(D) Filters outliers

42. How do you compute both mean and standard deviation simultaneously on grouped data?

(A) `df.groupby('dev').agg(['mean', 'std'])`
(B) `df.groupby('dev').calc('mean', 'std')`
(C) `df.groupby('dev').both()`
(D) `df.groupby('dev').describe()` only

43. Which method sorts a DataFrame by the values of one or more columns?

(A) df.order_by()
(B) df.sort_values()
(C) df.arrange()
(D) df.sort_by()

44. Which function joins two DataFrames based on a shared database key column?

(A) pd.concat()
(B) pd.merge()
(C) pd.append()
(D) pd.glue()

45. What is the default join type in `pd.merge(df1, df2, on='id')`?

(A) 'outer'
(B) 'inner'
(C) 'left'
(D) 'right'

46. What join type retains all rows from both DataFrames, filling missing matches with NaN?

(A) 'inner'
(B) 'outer'
(C) 'cross'
(D) 'union'

47. Which function stacks DataFrames along rows (axis=0) or columns (axis=1)?

(A) pd.merge()
(B) pd.concat()
(C) pd.join()
(D) pd.align()

48. Which method transforms wide-format tabular data into long-format tidy data?

(A) df.pivot()
(B) pd.melt()
(C) df.unstack()
(D) df.reshape()

49. Which method creates a spreadsheet-style pivot table with aggregation?

(A) df.crosstab()
(B) df.pivot_table()
(C) df.summarize()
(D) df.cross_pivot()

50. How do you parse a column of date strings into Pandas datetime format?

(A) `pd.to_datetime(df['timestamp'])`
(B) `df['timestamp'].as_date()`
(C) `pd.parse_time()`
(D) `pd.date_cast()`

51. How do you extract the month integer from a datetime column `df['date']`?

(A) `df['date'].month`
(B) `df['date'].dt.month`
(C) `df['date'].to_month()`
(D) `pd.get_month(df['date'])`

52. Which method downsamples or upsamples time-series data at regular frequency intervals?

(A) df.sample()
(B) df.resample()
(C) df.interpolate()
(D) df.retime()

53. What does `df.resample('1h').mean()` do to a 10 Hz telemetry dataset?

(A) Computes 1-hour hourly averages across all timestamps
(B) Filters out 1st hour
(C) Samples 1 row per hour
(D) Interpolates hourly

54. Which method computes the rolling moving average over a window of 50 samples?

(A) df.moving(50).mean()
(B) df.rolling(window=50).mean()
(C) df.sliding(50).mean()
(D) df.window(50).avg()

55. What does `df.ewm(span=20).mean()` calculate?

(A) Exponential Moving Average (EMA) with span 20
(B) Equal weight mean
(C) Rolling median
(D) Cumulative product

56. How do you perform vector string operations such as finding substrings in a Series?

(A) `df['str_col'].str.contains('pattern')`
(B) `df['str_col'].contains('pattern')`
(C) `pattern in df['str_col']`
(D) `df['str_col'].find('pattern')`

57. Which method changes column data types to category for memory optimization?

(A) `df['band'].astype('category')`
(B) `df['band'].to_cat()`
(C) `df['band'].categorize()`
(D) `pd.Categorical(df)`

58. What does `df.duplicated()` return?

(A) A DataFrame of unique rows
(B) A boolean Series indicating duplicated rows
(C) Count of duplicates
(D) Drops duplicates

59. Which method removes duplicate rows from a DataFrame?

(A) df.clean_duplicates()
(B) df.drop_duplicates()
(C) df.unique_rows()
(D) df.purge_dups()

60. How do you filter a DataFrame where voltage > 3.0 AND current < 0.5?

(A) `df[(df['voltage'] > 3.0) and (df['current'] < 0.5)]`
(B) `df[(df['voltage'] > 3.0) & (df['current'] < 0.5)]`
(C) `df[df.voltage > 3.0 || df.current < 0.5]`
(D) `df.where(v > 3.0, c < 0.5)`

61. Why does using the Python keyword `and` fail for element-wise boolean DataFrame filtering?

(A) Because `and` evaluates the truth value of the entire Series container as a single ambiguous truth, requiring bitwise `&`
(B) Pandas syntax error
(C) Python bug
(D) Keyword `and` is deprecated

62. Which method creates a secondary y-axis sharing the same x-axis on a Matplotlib plot?

(A) ax.secondary_y()
(B) ax.twinx()
(C) ax.twiny()
(D) ax.dual_axis()

63. What is the primary use case of `ax.twinx()` in electrical engineering plots?

(A) Plotting voltage and current on the same time scale with different measurement units and magnitudes
(B) Displaying 3D surfaces
(C) Comparing two frequencies
(D) Mirroring curves

64. Which function fills the area between two curves on a plot?

(A) plt.fill()
(B) plt.fill_between()
(C) plt.shade()
(D) plt.color_area()

65. What parameter in `plt.plot()` or `plt.scatter()` controls the transparency of points or lines?

(A) transparency
(B) alpha
(C) opacity
(D) blend

66. How do you plot error bars with standard deviation bars on data points?

(A) plt.error()
(B) plt.errorbar()
(C) plt.std_plot()
(D) plt.bars(with_error=True)

67. Which function creates a 2D pseudocolor heat map of a matrix on a rectangular grid?

(A) plt.heatmap()
(B) plt.pcolormesh()
(C) plt.color_grid()
(D) plt.raster()

68. Which function displays a 2D image or matrix with pixel interpolation?

(A) plt.render_image()
(B) plt.imshow()
(C) plt.picture()
(D) plt.pixel_plot()

69. Which function adds a color bar legend showing the mapping of values to colors?

(A) plt.legend()
(B) plt.colorbar()
(C) plt.palette()
(D) plt.cmap_bar()

70. How do you set a figure size to 10 inches wide by 6 inches high?

(A) `plt.figure(figsize=(10, 6))`
(B) `plt.size(10, 6)`
(C) `plt.dimensions(10, 6)`
(D) `plt.window(10, 6)`

71. Which function adjusts subplot spacing parameters so labels and tick marks do not overlap?

(A) plt.auto_space()
(B) plt.tight_layout()
(C) plt.clean_layout()
(D) plt.fit_plot()

72. What is the purpose of `plt.subplot2grid()` or `GridSpec` in Matplotlib?

(A) Enables complex multi-column and multi-row subplot layouts of varying spans and sizes
(B) Generates 3D CAD meshes
(C) Draws resistor grids
(D) Imports Excel sheets

73. Which function sets custom tick positions on the x-axis?

(A) plt.x_ticks()
(B) plt.xticks() or ax.set_xticks()
(C) plt.set_xmarks()
(D) plt.axis_ticks()

74. Which function sets custom text tick labels on the x-axis?

(A) plt.xticklabels() or ax.set_xticklabels()
(B) plt.x_names()
(C) plt.set_axis_labels()
(D) plt.xtext()

75. What does `plt.axhline(y=0, color='k', linestyle='--')` draw?

(A) A vertical marker line
(B) A horizontal dashed black reference line across the entire plot at y=0
(C) A diagonal asymptote
(D) A point marker

76. What causes the notorious `SettingWithCopyWarning` in Pandas?

(A) Modifying a DataFrame slice created by chained indexing (`df[a][b] = val`) where Pandas cannot determine if slice is view or copy
(B) Using deprecated numpy versions
(C) Writing to read-only disk
(D) Type mismatch during assignment

77. How is `SettingWithCopyWarning` completely avoided during assignment?

(A) Always use single `.loc[row_indexer, col_indexer] = val` indexing
(B) Disable warnings using filterwarnings
(C) Use Python for loops
(D) Cast to dict first

78. What major architectural feature in Pandas 2.0+ eliminates `SettingWithCopyWarning` entirely?

(A) Copy-on-Write (CoW) mode (`pd.options.mode.copy_on_write = True`)
(B) Automatic multithreading
(C) Discarding NumPy backend
(D) Disallowing slicing

79. What was the traditional internal memory manager in Pandas called, and why did it fragment memory?

(A) BlockManager, which grouped columns by dtype into 2D numpy blocks, causing massive memory duplication on single column modifications
(B) HeapManager
(C) PointerArray
(D) ColumnStacker

80. What is the primary advantage of Apache Arrow backend integration in Pandas 2.0 (`dtype_backend='pyarrow'`)?

(A) Zero-copy data sharing, native missing data handling, and SIMD string performance without Python object conversion
(B) Direct GPU rendering
(C) Automatic cloud sync
(D) Smaller font rendering

81. What does `df.eval()` and `df.query()` use internally to accelerate string-based DataFrame operations?

(A) Numexpr engine, evaluating expressions in chunks without allocating large intermediate memory arrays
(B) Regular expressions
(C) Python eval() function
(D) Subprocess execution

82. Why does `df['category_col'].astype('category')` dramatically reduce RAM consumption for repeated text?

(A) It replaces strings with small integer codes (int8/int16) and maintains a single small dictionary of unique category labels
(B) It zips the strings
(C) It drops duplicates
(D) It encrypts strings

83. What does `pd.MultiIndex.from_tuples()` construct?

(A) A hierarchical multi-level index along rows or columns
(B) A binary search index
(C) A primary database key
(D) A spatial coordinate tree

84. How do you select data from an inner level of a MultiIndex without dropping outer levels?

(A) `df.xs(key, level=1)` or `IndexSlice`
(B) `df.inner_loc()`
(C) `df.level_get()`
(D) `df.slice_level()`

85. What is the Artist hierarchy in Matplotlib's object-oriented architecture?

(A) Figure -> Canvas -> Axes -> Axis, Tick, Line2D, Text, Collections
(B) Window -> Screen -> Pixels
(C) Widget -> Layout -> View
(D) Chart -> Trace -> Marker

86. What is Matplotlib 'Blitting' used for in real-time engineering instruments (e.g. software oscilloscopes)?

(A) Redrawing only modified dynamic artists (lines) without re-rendering static backgrounds, achieving 60+ FPS animation
(B) Color dithering
(C) Anti-aliasing text
(D) Hardware 3D rendering

87. Which class from `matplotlib.animation` handles frame-based animated plotting?

(A) PlotAnimator
(B) FuncAnimation
(C) LiveFeed
(D) DynamicPlotter

88. How do you plot a 2D Fourier Spectrogram with time on x-axis and frequency on y-axis in Matplotlib?

(A) `plt.specgram(signal, Fs=fs, NFFT=256, noverlap=128)`
(B) `plt.spectrogram(signal)`
(C) `plt.fft2d(signal)`
(D) `plt.fourier_map(signal)`

89. Which function from `scipy.signal` calculates Short-Time Fourier Transform spectrogram matrices for Pandas DataFrames?

(A) `scipy.signal.spectrogram()`
(B) `scipy.signal.fft_table()`
(C) `scipy.signal.stft_grid()`
(D) `scipy.signal.waterfall()`

90. What is a Bode plot in frequency response analysis, and how is it structured in Matplotlib?

(A) Two vertically aligned subplots sharing x-axis: top plot is Magnitude in dB vs Log Frequency; bottom plot is Phase in degrees vs Log Frequency
(B) A 3D surface plot
(C) A polar circle diagram
(D) A time-domain envelope plot

91. Which command sets logarithmic spacing on the horizontal frequency axis in Matplotlib?

(A) `ax.set_xscale('log')` or `plt.semilogx()`
(B) `ax.log_axis(0)`
(C) `plt.x_log10()`
(D) `ax.scale_x('exponential')`

92. How do you plot eye diagrams in digital communications to evaluate Intersymbol Interference (ISI)?

(A) Folding periodic signal segments of 2 symbol intervals overlaid on top of each other using Matplotlib
(B) Computing FFT phase
(C) Plotting constellation points
(D) Using polar coordinates

93. Which function in Matplotlib generates a polar coordinate plot (e.g. for antenna radiation patterns)?

(A) `plt.subplot(projection='polar')` or `plt.polar()`
(B) `plt.circle_plot()`
(C) `plt.azimuth_plot()`
(D) `plt.radial()`

94. What does `df.memory_usage(deep=True)` reveal compared to `df.memory_usage()`?

(A) Includes the actual memory allocated by Python string objects referenced by pointers in object dtypes
(B) Scans disk storage
(C) Includes operating system cache
(D) Shows virtual RAM

95. What is the difference between `df.apply()` and `df.map()` / `df.applymap()`?

(A) `apply()` operates along DataFrame axes (rows or columns); `map()` operates element-wise on a Series; `applymap()` (now `map`) operates element-wise on DataFrames
(B) No difference
(C) apply() only works on numbers
(D) map() is for geographic data

96. Why is `df.apply()` with a Python lambda generally much slower than vectorized NumPy operations?

(A) It executes an interpreted Python function loop over every row, incurring CPython function invocation overhead
(B) It runs on a single thread
(C) It converts floats to strings
(D) It duplicates the DataFrame

97. Which method computes the cross-tabulation frequency of two factors in Pandas?

(A) pd.cross_tab()
(B) pd.crosstab()
(C) pd.frequency_table()
(D) pd.contingency()

98. How do you configure Matplotlib to use LaTeX font rendering for publication-quality engineering papers?

(A) `plt.rcParams.update({'text.usetex': True, 'font.family': 'serif'})`
(B) `plt.latex_mode(True)`
(C) `plt.enable_tex()`
(D) `plt.font('latex')`

99. What parameter in `pd.read_csv()` allows reading extremely large multi-gigabyte telemetry files in chunks?

(A) `chunksize=N`
(B) `batch_size=N`
(C) `stream_rows=N`
(D) `iter_chunk=N`

100. What does iterating over `pd.read_csv('telemetry.csv', chunksize=10000)` yield?

(A) An iterator of DataFrames, each containing 10,000 rows
(B) A list of lists
(C) A memory-mapped array
(D) A dictionary of rows

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the core internal architecture of Pandas Series and DataFrames, detailing index alignment and columnar memory layout.

Q2. Contrast label-based indexing `.loc` with integer-position-based indexing `.iloc` in Pandas with boundary condition examples.

Q3. Analyze the `SettingWithCopyWarning` in Pandas: root causes, memory hazards, and the Copy-on-Write (CoW) solution in Pandas 2.0+.

Q4. Explain data aggregation and transformation using `groupby()`, `agg()`, `transform()`, and `apply()` in Pandas.

Q5. Demonstrate handling missing telemetry data in Pandas: detection, deletion strategies, and forward/backward/interpolation imputation.

Q6. Explain merging, joining, and concatenation of DataFrames with relational database semantics (Inner, Outer, Left, Right).

Q7. Demonstrate reshaping operations in Pandas: `pivot()`, `pivot_table()`, `melt()`, and multi-index stacking.

Q8. Detail time-series analysis in Pandas: DatetimeIndex, frequency strings, shifting, rolling statistics, and resampling.

Q9. Explain memory optimization strategies in Pandas: categorical data types, numeric downcasting, and PyArrow backends.

Q10. Describe the object-oriented architecture of Matplotlib: Figure, Axes, Axis, Tick, and Artist hierarchy.

Q11. Demonstrate plotting twin axes (`ax.twinx()`) for dual-parameter engineering plots with distinct units and scales.

Q12. Explain how to construct a complete Bode Plot in Matplotlib showing Magnitude in dB and Phase in degrees.

Q13. Demonstrate plotting a Short-Time Fourier Transform (STFT) Spectrogram using `plt.pcolormesh` and custom colormaps.

Q14. Explain Matplotlib animation via `FuncAnimation` and the concept of 'Blitting' for high-refresh oscilloscope displays.

Q15. Demonstrate reading multi-gigabyte CSV telemetry files in chunks using `chunksize` and building an aggregated summary.

Q16. Explain how to plot 3D surface and wireframe plots in Matplotlib using `mpl_toolkits.mplot3d` for antenna beam patterns.

Q17. Demonstrate custom date formatting and locator configuration on Matplotlib time axes using `matplotlib.dates`.

Q18. Explain the use of `GridSpec` in Matplotlib for creating complex dashboard layouts with asymmetric subplots.

Q19. Detail string manipulation techniques in Pandas using `.str` accessor methods (Regex extraction, splitting, replacing).

Q20. Explain hierarchical MultiIndex operations in Pandas: creation, slicing with `IndexSlice`, and cross-sections (`.xs`).

Q21. Implement an automated end-to-end data pipeline in Pandas to clean, calibrate, and filter IoT wireless sensor node packets.

Q22. Write a Python script using Matplotlib to plot an Eye Diagram for evaluating Intersymbol Interference in PAM-4 signals.

Q23. Implement a custom publication-grade Matplotlib style sheet programmatically using `matplotlib.rcParams`.

Q24. Explain how to plot antenna radiation patterns on polar coordinates using `matplotlib.pyplot.polar`.

Q25. Implement an automated anomaly detection system on streaming time-series in Pandas using Exponential Moving Averages.

Q26. Write a Python script to plot a Smith Chart impedance visualization using Matplotlib.

Q27. Explain how to export interactive web visualizations using Plotly and compare performance against static Matplotlib.

Q30. Implement an automated PDF report generation pipeline using Matplotlib `PdfPages` combining multi-page test results.

Q28. Demonstrate optimizing Pandas queries on 50-million-row DataFrames using Dask or Polars lazy evaluation.

Q29. Explain custom colormap generation in Matplotlib for high-contrast thermal imaging and microwave field intensity maps.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[B]	**Q21:**	[B]	**Q41:**	[B]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[B]	**Q22:**	[B]	**Q42:**	[A]	**Q62:**	[B]	**Q82:**	[A]
Q3:	[B]	**Q23:**	[B]	**Q43:**	[B]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[B]	**Q24:**	[B]	**Q44:**	[B]	**Q64:**	[B]	**Q84:**	[A]
Q5:	[B]	**Q25:**	[B]	**Q45:**	[B]	**Q65:**	[B]	**Q85:**	[A]
Q6:	[B]	**Q26:**	[B]	**Q46:**	[B]	**Q66:**	[B]	**Q86:**	[A]
Q7:	[B]	**Q27:**	[A]	**Q47:**	[B]	**Q67:**	[B]	**Q87:**	[B]
Q8:	[A]	**Q28:**	[B]	**Q48:**	[B]	**Q68:**	[B]	**Q88:**	[A]
Q9:	[B]	**Q29:**	[B]	**Q49:**	[B]	**Q69:**	[B]	**Q89:**	[A]
Q10:	[B]	**Q30:**	[B]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[B]	**Q31:**	[B]	**Q51:**	[B]	**Q71:**	[B]	**Q91:**	[A]
Q12:	[B]	**Q32:**	[A]	**Q52:**	[B]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[B]	**Q33:**	[B]	**Q53:**	[A]	**Q73:**	[B]	**Q93:**	[A]
Q14:	[B]	**Q34:**	[B]	**Q54:**	[B]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[B]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[B]	**Q95:**	[A]
Q16:	[B]	**Q36:**	[B]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[B]	**Q37:**	[B]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[B]
Q18:	[A]	**Q38:**	[B]	**Q58:**	[B]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[B]	**Q39:**	[B]	**Q59:**	[B]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[B]	**Q40:**	[B]	**Q60:**	[B]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

view or an independent copy. If it returns a copy, assigning `['voltage'] = 3.3` modifies that temporary copy which is immediately discarded, leaving the original `df` completely unmodified!

Best Practice Solution:
Use single-step `.loc` assignment:
`df.loc[df['ch'] == 1, 'voltage'] = 3.3`

Pandas 2.0+ Copy-on-Write (CoW) Paradigm:
Under CoW (`pd.options.mode.copy_on_write = True`), any DataFrame slice created is guaranteed to be a shallow view until a write is attempted. When a write occurs, Pandas lazily allocates a copy of that specific column only, completely eradicating ambiguity and eliminating `SettingWithCopyWarning`.

- `df.dropna(how='any')`: Drops rows if any channel has missing data.
- `df.dropna(subset=['critical_channel'])`: Drops only when key telemetry is lost.

3. Imputation Techniques:
- Constant Filling: `df.fillna(0.0)`
- Forward Fill (`ffill`): Propagates the last valid measured reading forward in time, matching physical sample-and-hold circuit behavior: `df.ffill()`.
- Backward Fill (`bfill`): Propagates the next valid future sample backward: `df.bfill()`.
- Polynomial and Spline Interpolation:
`df['voltage'].interpolate(method='time')` fits smooth mathematical curves across missing timestamps.

---

3. `pivot_table()`:
Handles duplicate index-column combinations by applying an aggregation function (e.g. `aggfunc='mean'`).

>>Solution Question Bank: Real-time series analysis in Database Data Pandas extension data types in time rolling statistics and PyArrow backends.

>>**Solution Statement:** Define the object to interact with the current figure. Modify figure properties like axes, ticks, and limits.and scales.

```

---



>>Solution Question 18: To construct a Short-Time Fourier Transform (STFT) in MATLAB, you can use the `spectrogram` function. Here's an example code snippet: ```
```

```
# Linear chirp from 500 Hz to 3000 Hz
x = signal.chirp(t, f0=500, t1=2, f1=3000, method='linear')

# Compute STFT
f, t_spec, Sxx = signal.spectrogram(x, fs, nperseg=256, noverlap=128)

fig, ax = plt.subplots(figsize=(9, 5))
pcm = ax.pcolormesh(t_spec, f, 10 * np.log10(Sxx + 1e-12), shading='gouraud', cmap='viridis')
ax.set_title('Linear Chirp Signal Spectrogram')
ax.set_xlabel('Time (seconds)')
ax.set_ylabel('Frequency (Hz)')
cbar = fig.colorbar(pcm, ax=ax)
cbar.set_label('Power Spectral Density (dB/Hz)')
```

```
  # Filter corrupted packets in current chunk
  valid_chunk = chunk[chunk['crc_valid'] == True]

  # Accumulate running statistics
  total_records += len(valid_chunk)
  sum_voltage += valid_chunk['voltage_v'].sum()
  max_temperature = max(max_temperature, valid_chunk['temp_c'].max())

  mean_voltage = sum_voltage / total_records
  ```
```

---



**>>Solution Question 18** Instructor 3D surface and wireframe plots in MATLAB using `meshgrid` and `surf` for potential patterns.
```

```
  # Major ticks every 2 hours, minor ticks every 30 minutes
  ax.xaxis.set_major_locator(mdates.HourLocator(interval=2))
  ax.xaxis.set_minor_locator(mdates.MinuteLocator(interval=30))

  # Format major ticks as 'HH:MM AM/PM'
  ax.xaxis.set_major_formatter(mdates.DateFormatter('%I:%M %p'))

  # Rotate tick labels to prevent overlap
  fig.autofmt_xdate()
```

```
# 2. Case conversion and whitespace stripping
df['status_clean'] = parsed['status'].str.strip().str.upper()

# 3. String replacement
df['port_num'] = parsed['port'].str.replace('COM', '')
```

>>Solution to Question 28 Implement a custom middleware component in Python that logs all incoming HTTP requests, including the client's IP address, the requested URL, and the response status code.

```
# 3. Handle outliers using IQR thresholding per node
def remove_iqr_outliers(group):
q25 = group['raw_val'].quantile(0.25)
q75 = group['raw_val'].quantile(0.75)
iqr = q75 - q25
valid_mask = (group['raw_val'] >= q25 - 1.5 * iqr) & (group['raw_val'] <= q75 + 1.5 * iqr)
group.loc[~valid_mask, 'raw_val'] = np.nan
return group

df = df.groupby('node_id', group_keys=False).apply(remove_iqr_outliers)

# 4. Interpolate missing values
df['calibrated_temp'] = df.groupby('node_id')['raw_val'].transform(
lambda s: s.interpolate(method='time').ffill().bfill()
) * 0.01 - 50.0 # Linear calibration formula

return df
```

Solution Strategy: Python custom unit conversion and interpolation.

Question 28: Explain the role of the Eye Diagram in evaluating the signal integrity of a PAM-4 signal.

IEEE / Nature Publication Style Sheet Configuration:

```python
import matplotlib.pyplot as plt

def apply_publication_theme():
params = {
'font.family': 'sans-serif',
'font.sans-serif': ['Helvetica', 'Arial', 'DejaVu Sans'],
'font.size': 10,
'axes.labelsize': 11,
'axes.titlesize': 12,
'xtick.labelsize': 9,
'ytick.labelsize': 9,
'legend.fontsize': 9,
'figure.titlesize': 14,
'lines.linewidth': 1.5,
'lines.markersize': 5,
'axes.linewidth': 1.0,
'axes.grid': True,
'grid.linestyle': '--',
'grid.alpha': 0.6,
'savefig.dpi': 300,
'savefig.bbox': 'tight'
}
plt.rcParams.update(params)
```

>> Solution to Question 24: Explain how to plot antenna radiation patterns on polar coordinates using `matplotlib.pyplot.polar`.

I

>>Solution Statement:Write a Python script to plan a Satellite mission using Exponential Moving Averages.

```
  center_x = r / (r + 1.0)
  radius = 1.0 / (r + 1.0)
  ax.plot(center_x + radius * np.cos(theta), radius * np.sin(theta), 'b--', alpha=0.5)

  ax.set_title('Normalized Smith Chart (Impedance)')
  ax.grid(True, linestyle=':')
  return fig, ax
```

```
ax2.plot(test_data['freq'], test_data['s21'], 'b-', label='S21 Gain')
ax2.set_title('S21 Transmission Characteristic')
ax2.set_xlabel('Frequency (GHz)')
ax2.set_ylabel('Magnitude (dB)')
ax2.grid(True)
pdf.savefig(fig2)
plt.close(fig2)
```

>>Solution to Question 28 Generate optimized Gerber files for a 4-layer PCB with controlled impedance traces and export drill files in Excellon format.

```
field_intensity = np.random.rand(100, 100)
plt.imshow(field_intensity, cmap=thermal_cmap)
plt.colorbar(label='Field Power (W/m^2)')
plt.title('High-Contrast Thermal Map')
```
